Exercise 12

AIM :

To write a C program for below requirements

1.Create a Stack using linked list with push(), pop() functions, also use function to check whether it is full or empty

2.Create a Queue using list with push() and pop()

ALGORITHM :

Step 1: Start the Program

1. Begin program execution.

Step 2: Define Common Node Structure

2. Define a Node structure containing:
    - Integer data
    - Pointer next to the next node

Step 3: Define Stack Structure

3. Define a Stack structure with:
    - Pointer top
    - Integer size
    - Integer capacity (unlimited)

Step 4: Define Queue Structure

4. Define a Queue structure with:
    - Pointer front
    - Pointer rear
    - Integer size

Step 5: Declare Stack and Queue Functions

5. Declare functions for:
    - Stack creation, push, pop, display, empty check, and memory deallocation

- Queue creation, enqueue, dequeue, display, empty check, and memory deallocation

Step 6: Initialize Stack and Queue Pointers

6. Initialize:
    - myStack = NULL
    - myQueue = NULL

Step 7: Display Main Menu in a Loop

7. Repeat the following until the user exits:
    - Display main menu:
        1. Stack Operations
        2. Queue Operations
        3. Exit
    - Read user choice.

Step 8: Perform Stack Operations

8. If the user selects **Stack Operations**:
    - Create a new stack if it does not exist.
    - Display stack submenu:
        1. Push
        2. Pop
        3. Check if Empty
        4. Display Stack
        5. Back to Main Menu

Stack Operation Steps

Push Operation

9. Allocate memory for a new node.
10. Store data in the node.
11. Set newNode->next = top.
12. Update top to the new node.
13. Increment stack size.

Pop Operation

14. Check if the stack is empty.

15. If not empty:
    - Store top node data.
    - Move top to the next node.
    - Free the removed node.
    - Decrement stack size.
    - Return popped value.

Check if Stack is Empty

16. If top == NULL, display stack is empty.
17. Otherwise, display stack is not empty.

Display Stack

18. Traverse from top to NULL.
19. Print elements from top to bottom.

Step 9: Perform Queue Operations

20. If the user selects **Queue Operations**:

- Create a new queue if it does not exist.
- Display queue submenu:
    1. Enqueue
    2. Dequeue
    3. Check if Empty
    4. Display Queue
    5. Back to Main Menu

Queue Operation Steps

Enqueue Operation

21. Allocate memory for a new node.
22. Store data and set next = NULL.
23. If queue is empty:
    - Set both front and rear to new node.
24. Otherwise:
    - Link new node at the end.
    - Update rear.

25. Increment queue size.

Dequeue Operation

26. Check if the queue is empty.
27. If not empty:
    - Store front node data.
    - Move front to the next node.
    - If queue becomes empty, set rear = NULL.
    - Free removed node.
    - Decrement queue size.
    - Return dequeued value.

Check if Queue is Empty

28. If front == NULL, display queue is empty.
29. Otherwise, display queue is not empty.

Display Queue

30. Traverse from front to rear.
31. Print elements from front to rear.

Step 10: Exit the Program

32. If user selects Exit:
    - Free all stack nodes.
    - Free all queue nodes.
    - Release allocated memory.
    - Terminate the program.

Step 11: Stop

33. End program execution.

OUTPUT :

```
PS D:\C programming\DHAVAMANI_EX12> gcc main.c -o main
PS D:\C programming\DHAVAMANI_EX12> ./main

========== MAIN MENU ==========
1. Stack Operations
2. Queue Operations
3. Exit
Enter your choice: 1
New stack created

--- Stack Operations ---
1. Push
2. Pop
3. Check if Empty
4. Display Stack
5. Back to Main Menu
Enter operation choice: 1
Enter value to push: 45
Pushed 45 to stack

--- Stack Operations ---
1. Push
2. Pop
3. Check if Empty
4. Display Stack
5. Back to Main Menu
Enter operation choice: 1
Enter value to push: 55
Pushed 55 to stack
```

```
--- Stack Operations ---
1. Push
2. Pop
3. Check if Empty
4. Display Stack
5. Back to Main Menu
Enter operation choice: 1
Enter value to push: 65
Pushed 65 to stack

--- Stack Operations ---
1. Push
2. Pop
3. Check if Empty
4. Display Stack
5. Back to Main Menu
Enter operation choice: 4
Stack (Top to Bottom): 65 -> 55 -> 45

--- Stack Operations ---
1. Push
2. Pop
3. Check if Empty
4. Display Stack
5. Back to Main Menu
Enter operation choice: 2
Popped value: 65

--- Stack Operations ---
1. Push
2. Pop
3. Check if Empty
4. Display Stack
5. Back to Main Menu
Enter operation choice: 2
Popped value: 55
```

```
--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Check if Empty
4. Display Queue
5. Back to Main Menu
Enter operation choice: 1
Enter value to enqueue: 55
Enqueued 55 to queue

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Check if Empty
4. Display Queue
5. Back to Main Menu
Enter operation choice: 1
Enter value to enqueue: 45
Enqueued 45 to queue

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Check if Empty
4. Display Queue
5. Back to Main Menu
Enter operation choice: 1
Enter value to enqueue: 35
Enqueued 35 to queue
```

```
--- Stack Operations ---
1. Push
2. Pop
3. Check if Empty
4. Display Stack
5. Back to Main Menu
Enter operation choice: 2
Popped value: 45

--- Stack Operations ---
1. Push
2. Pop
3. Check if Empty
4. Display Stack
5. Back to Main Menu
Enter operation choice: 3
Stack is empty

--- Stack Operations ---
1. Push
2. Pop
3. Check if Empty
4. Display Stack
5. Back to Main Menu
Enter operation choice: 5

========== MAIN MENU ==========
1. Stack Operations
2. Queue Operations
3. Exit
Enter your choice: 2
New queue created
```

```
--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Check if Empty
4. Display Queue
5. Back to Main Menu
Enter operation choice: 4
Queue (Front to Rear): 55 -> 45 -> 35

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Check if Empty
4. Display Queue
5. Back to Main Menu
Enter operation choice: 2
Dequeued value: 55

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Check if Empty
4. Display Queue
5. Back to Main Menu
Enter operation choice: 2
Dequeued value: 45

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Check if Empty
4. Display Queue
5. Back to Main Menu
Enter operation choice: 3
Queue is empty

--- Queue Operations ---
1. Enqueue
2. Dequeue
3. Check if Empty
4. Display Queue
5. Back to Main Menu
Enter operation choice: 5

========== MAIN MENU ==========
1. Stack Operations
2. Queue Operations
3. Exit
Enter your choice: 3

Exiting program...
Stack memory freed
Queue memory freed
```